ToddySoft Connect S7 Web API - Apache StreamPipes

An Apache StreamPipes **adapter** for Siemens S7-1200 and S7-1500 controllers through their Web API, built on the ToddySoft Connect S7 Web API driver. It reads tags from Siemens S7-1200 and S7-1500 controllers through the controller's own Web API (JSON-RPC over HTTPS). Every polling interval produces one event in StreamPipes with the selected tags' values.

The adapter comes as its own **extension service**: a container that runs next to your StreamPipes installation, registers with it, and appears in StreamPipes as the adapter *ToddySoft Connect S7 Web API*.

Requirements

- **Apache StreamPipes 0.99**, installed with Docker (the standard installer). The service joins the installation's Docker network.
- **Linux on the machine running StreamPipes**, amd64 or arm64, with a route to the devices. Docker Desktop (macOS, Windows) runs containers in a virtual machine: adapters work with *Enter address*, but no search reaches the plant network.
- A **ToddySoft licence for S7 Web API** for production use. Without one the service runs on its bundled trial licence until that expires.

Installation

Loading the image

The service is delivered as an image archive, one per architecture, in the share you received your licence and downloads from: `streampipes-s7-webapi-<version>-<architecture>.tar.gz`, for `amd64` or `arm64`. Use the archive of the version you were sent and the architecture of the machine (`uname -m`: `x86_64` is amd64, `aarch64` is arm64).

```
docker load < streampipes-s7-webapi-1.3.0-amd64.tar.gz
```

> **The images are not on Docker Hub or any public registry**, and the same goes for the discovery agent: every image reaches you through this one channel. We plan to move to a private registry with a pull account per customer, so that `docker pull` and updates work without archives; until then, load each new version the same way.

Running the service

Start the service in the Docker network of your StreamPipes installation (`spnet` for the standard installer; `docker network ls` shows it):

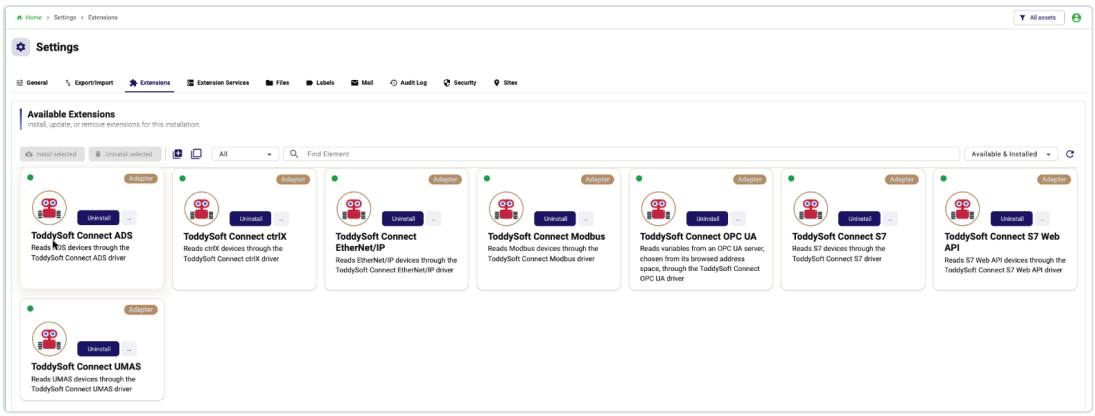
```
docker run -d --name toddysoft-s7-webapi --restart unless-stopped \
  --network spnet \
  -e SP_HOST=toddysoft-s7-webapi -e SP_PORT=8005 \
  -e SP_CLIENT_SECRET=<the SP_CLIENT_SECRET of your installation> \
  -e SP_PRIORITIZED_PROTOCOL=kafka \
  toddysoft/streampipes-s7-webapi:1.3.0
```

- **`SP_HOST`** must be the container's own name: StreamPipes calls the service back by it.
- **`SP_CLIENT_SECRET`** must be the value the StreamPipes core runs with (the installer's `.env`). With another value the service cannot register.
- **`SP_PRIORITIZED_PROTOCOL`** is your installation's broker: `kafka` for the standard installer.
- Keep the service on the installation's network only. It reaches the core and the broker there by name; host networking would lose them. The devices are reached through the machine's routing, and the search through the discovery agent (see Reaching the devices).
- Files the adapter needs - a keystore, a trust store - are mounted with `-v`, for example `-v /opt/toddysoft/certs:/certs:ro`, and given in the form as `/certs/<file>`.
- PROFINET-DCP, the search this driver uses, sends and captures raw frames. The image carries the capture library; Docker grants `NET_RAW` by default, so add `--cap-add NET_RAW` only where capabilities are dropped.

`docker logs toddysoft-s7-webapi` should show the service registering with StreamPipes within a minute.

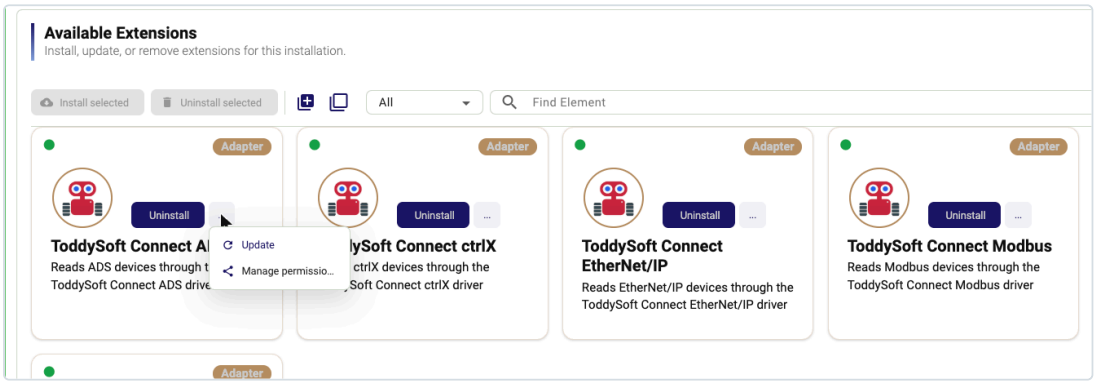
Installing the adapter in StreamPipes

Once the service has registered, install its adapter once: **Settings** → **Extensions**, find *ToddySoft Connect S7 Web API* and press **Install**. It then appears under **Connect** → **New Adapter**.



Updating

1. Load the new archive and start the new version in place of the old container (`docker rm -f toddysoft-s7-webapi`, then the `docker run` above with the new version).
2. **Update the installed adapter**: Settings → Extensions → the adapter's **"..."** → **Update**. StreamPipes keeps the adapter description it stored at installation, so without this step the form of the previous version is shown, and new settings are missing.

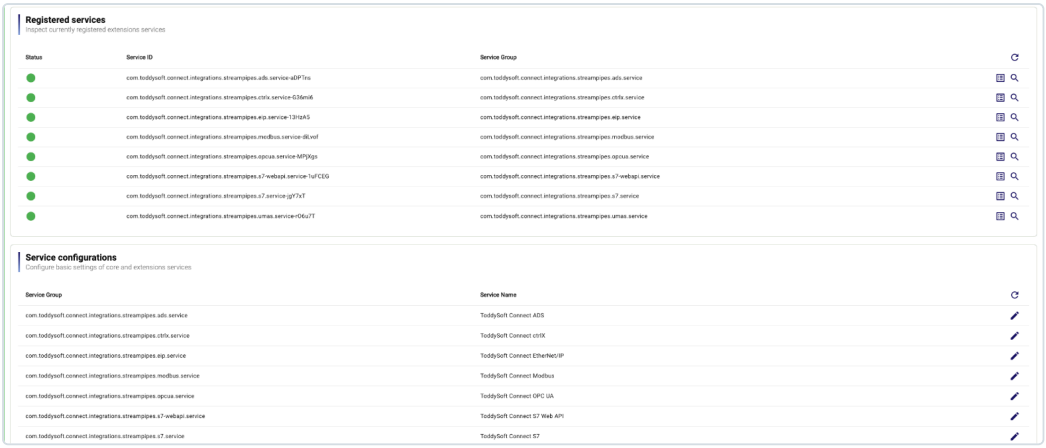


Adapters saved with the old version keep running after an update.

Licence

The driver reads its licence from the service's configuration in StreamPipes:

1. Open **Settings → Extension Services**. *Registered services* lists the running ToddySoft services; **Service configurations** below holds one entry per service.



2. Press the pencil next to *ToddySoft Connect S7 Web API*. The entry has one field, `licence-s7-webapi`.



3. Paste the licence file **Base64-encoded**: `base64 -i s7-webapi.lic` (macOS) or `base64 -w0 s7-webapi.lic` (Linux) prints it.

4. **Click into the field once more after pasting**, then press **Update**. StreamPipes takes the field's value only on that click; saving straight after pasting stores the previous value, usually empty.

The licence takes effect on the next browse or adapter start; no restart is needed. Without a licence, or with one the driver rejects, the bundled trial applies until it expires. The value is masked in the form but stored in StreamPipes' database as entered, so limit who can administer StreamPipes.

Reaching the devices

A service in a container sees only its container networks. Connecting, browsing and reading go out through the machine's routing and work as long as the machine has a route to the device. **The search** does not: broadcasts, multicast and PROFINET-DCP never leave the container's network, so on its own the search finds nothing on the plant network. There are two ways to give it the plant network.

The ToddySoft discovery agent (recommended)

The agent is a small container that runs once per machine with the host's network and searches the plant networks for every ToddySoft service that asks. It only searches: the adapter still connects from its own container. It comes as an archive next to this service's, `discovery-agent-<version>-<architecture>.tar.gz`; use the agent of the same version.

```
docker load < discovery-agent-1.3.0-amd64.tar.gz
docker run -d --name toddysoft-discovery-agent --restart unless-
        stopped \
  --network host --cap-add NET_RAW \
  toddysoft/discovery-agent:1.3.0
```

- **Nothing to set on the service.** At every search it looks for the agent at its container network's gateway, which on Docker is the machine itself. An agent started later is found on the next Reload. The search's status line says *via discovery agent* when the agent answered, *no discovery agent found · container networks only* when there is none, or why the agent refused.
- **Licence.** The service asks only for S7 Web API devices and sends the licence its driver runs on over an encrypted connection; the agent checks it, uses it for that one search and keeps nothing.
- **Security.** The agent listens only on the machine's Docker-network addresses (TCP 47900) and accepts connections only from the container networks; allow that port from them if the machine has a firewall. For more, set a shared secret: `-e TODDYSOFT_AGENT_SECRET=<secret>` on the agent and `-e TODDYSOFT_DISCOVERY_AGENT_SECRET=<secret>` on the service.
- **Discovering is not reaching.** A device the agent finds on a network the machine cannot route to is listed and then fails to connect.

A second network on the plant interface

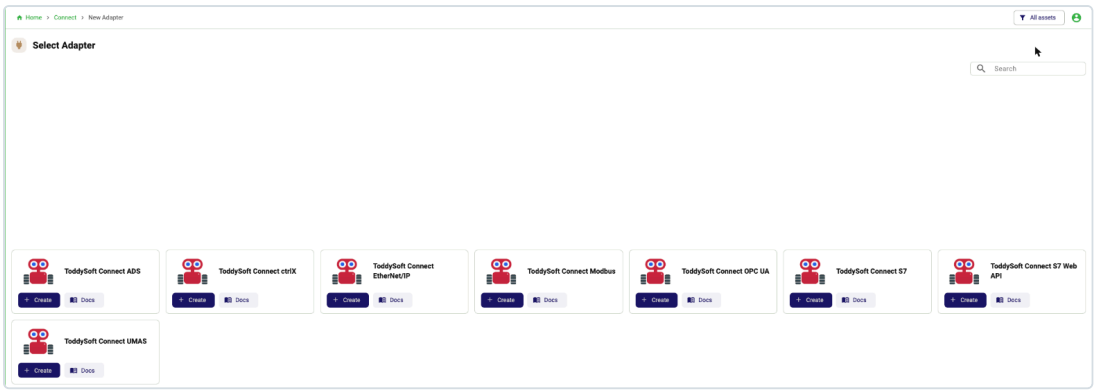
Alternatively, connect the service to a macvlan network on the plant interface, in the devices' own subnet, as a **second** network (it keeps `spnet`):

```
docker network create -d macvlan --subnet 192.168.23.0/24 --gateway
        192.168.23.1 -o parent=eth1 plant
docker network connect --ip 192.168.23.250 plant toddysoft-s7-webapi
```

Each service needs this, once per plant subnet. Set `-e TODDYSOFT_DISCOVERY_AGENT=off` on a service whose only network is a macvlan, so it does not look for an agent at the plant router.

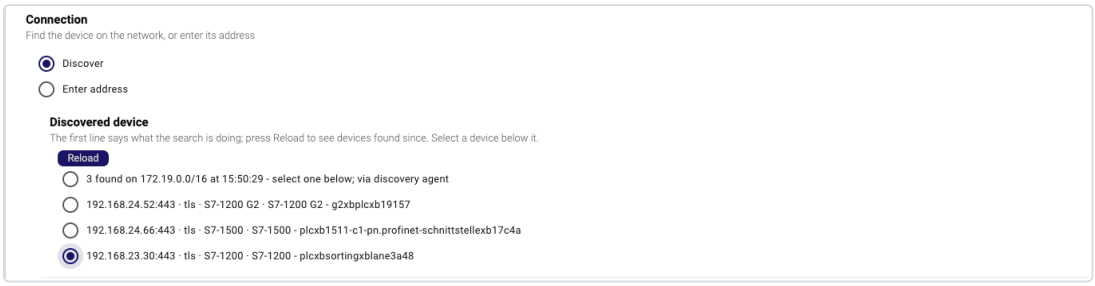
Creating an adapter

Open **Connect** → **New Adapter** and press **Create** on *ToddySoft Connect S7 Web API*.



Finding the device

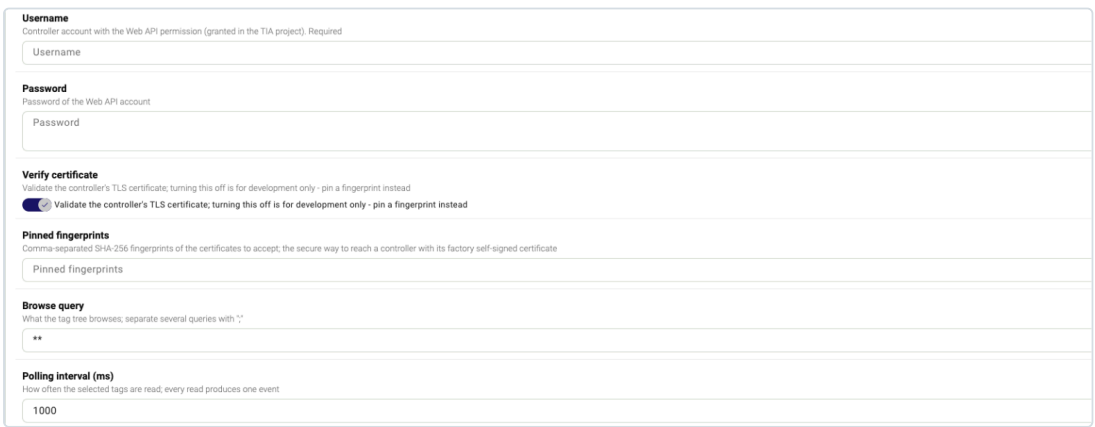
The form opens on **Connection → Discover**, and the search starts on its own.



The search uses PROFINET-DCP and lists every controller that answers, with its address, model and name, on port 443. Whether a controller's Web API is switched on cannot be seen from the search; that is decided in its TIA project.

The first line of **Discovered device** says what the search is doing and which source answered (*via discovery agent*, or the container's own networks). It runs in the background for up to 25 seconds, so press **Reload** next to the field to see the devices found since. Select a device below that line: its address and connection settings are taken as found. A finished search is reused for a minute; Reload after that searches again. To type the address instead, choose **Enter address**.

The Web API account and the certificate



- **Username** and **Password** are required: an account of the controller's user management with the Web API permission, granted in the TIA project. They are sent as a credential, never in the connection string.
- The controller presents its factory **self-signed certificate**. Keep **Verify certificate** on and pin the certificate instead:

  1. Choose the device, fill in the account and open the tag tree. The tree shows an error: *the device's TLS certificate is not trusted*, followed by the controller's SHA-256 fingerprint. The service's log names it too:

     ```
     docker logs toddysoft-s7-webapi 2>&1 | grep 'SHA-256
             fingerprint'
     ```

  2. Check it against the controller, paste it into **Pinned fingerprints** and press **Reload** next to the tag tree.

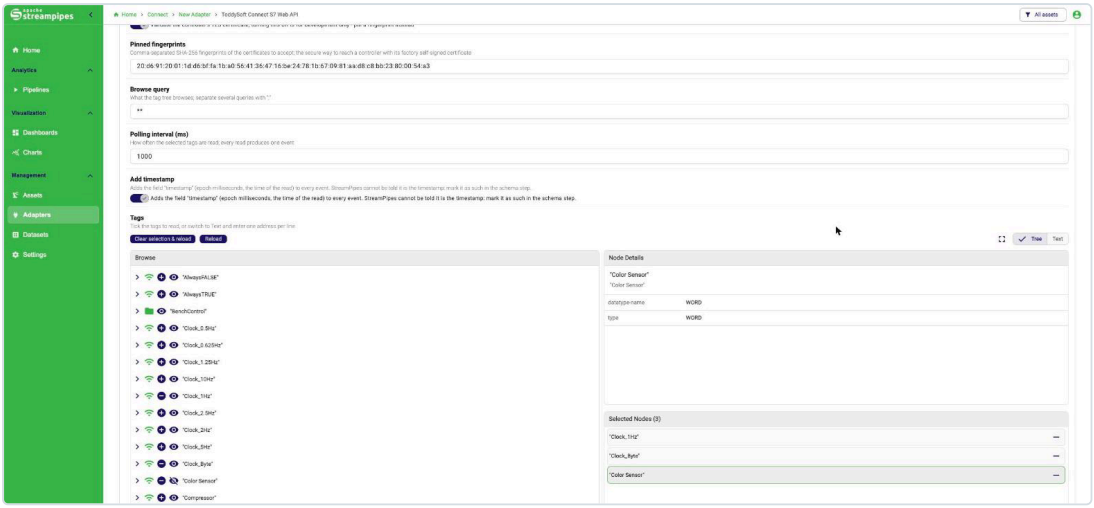
  Switching *Verify certificate* off also connects, but accepts any certificate; it is for development only.

Settings

Setting	Meaning
Connection	*Discover* a device, or *Enter address* to type the fields below yourself
Host, Port	Address of the controller; port 443
Username	Account with the Web API permission. Required
Password	Password of that account
Verify certificate	Check the controller's certificate; keep it on and pin the fingerprint
Pinned fingerprints	SHA-256 fingerprints to accept, comma-separated; see above
Browse query	What the tag tree shows, default `**`; separate several with `;`
Polling interval (ms)	How often the selected tags are read; every read produces one event
Add timestamp	Adds the field `timestamp` (the time of the read) to every event; see below
Tags	What is read; see below

The **Documentation** link at the top of the form opens a short version of this page in StreamPipes.

Choosing the tags



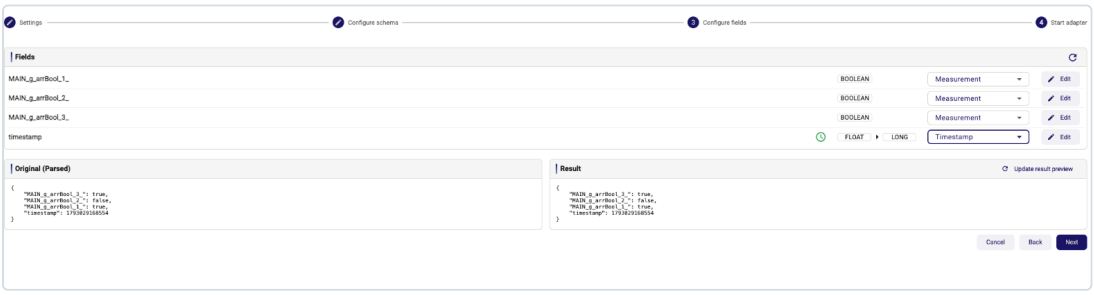
The tag tree shows what the **Browse query** returns. Press **+** next to a tag to read it, or on a folder to read everything below it; the eye shows a tag's type on the right, and **Selected Nodes** lists what will be read. **Reload** asks the device again; **Clear selection & reload** also empties the selection. Each tag becomes one field of the event, named after its address.

The tree's **Text** mode edits the selection directly, one address per line - useful to paste a list of addresses. StreamPipes puts its own comment lines into that editor (`# Provide OPC UA Node IDs …`); lines starting with `#` are ignored.

Events and the timestamp

Press **Next**. StreamPipes reads the tags once and shows the sample event; every poll then produces one event like it, with one field per tag that could be read and **`timestamp`**, the time of the read in milliseconds since the epoch.

StreamPipes cannot be told by the adapter which field is the timestamp. In **Configure fields**, set `timestamp`'s role to **Timestamp** and its type to **Long**:



- StreamPipes 0.99 types **every number as a float** in the schema it infers. The events themselves carry the exact values; change a field's type here where it matters.
- A tag that is itself named `timestamp` arrives as `timestamp_tag`.
- **Add timestamp** off leaves the field out, for a pipeline that sets its own.

Then name the adapter and start it under **Start adapter**.

Troubleshooting

- **The form looks like an older version, or a setting described here is missing**: update the installed adapter (Settings → Extensions → "…" → Update).
- **The tag tree shows an error**: the message names the cause - an unreachable address, a refused login, a missing licence. Where it says only *An unknown error has occurred*, the service's log has the details: `docker logs toddysoft-s7-webapi`.
- **The search finds nothing**: read its status line. *Container networks only* means neither the discovery agent nor a plant network is there (see Reaching the devices).
- **The licence is not taken**: click into the field after pasting, then Update. A licence the driver rejects is logged as `blob rejected for driver`, and the trial applies.
- **The service does not appear under Extensions**: check `docker logs toddysoft-s7-webapi`; usually `SP_CLIENT_SECRET` or the network is wrong.
- **The tag tree says the device's TLS certificate is not trusted**: pin the fingerprint it names (see above).
- **"Login failed for user '…'"**: the account or its password is wrong, or the account lacks the Web API permission in the TIA project.

Version history

Unreleased — 1.3.0

Built on ToddySoft Connect 1.3.0.

- **New:** the first release of the S7 Web API adapter for Apache StreamPipes 0.99 - device search, the tag tree, polling into StreamPipes events, the licence in the service configuration, and search from containers through the ToddySoft discovery agent.